

# Introduction to Containerization

**Faculty of Technology - University of Sri Jayewardenepura**

Hansini Gunasena

Visiting Lecturer

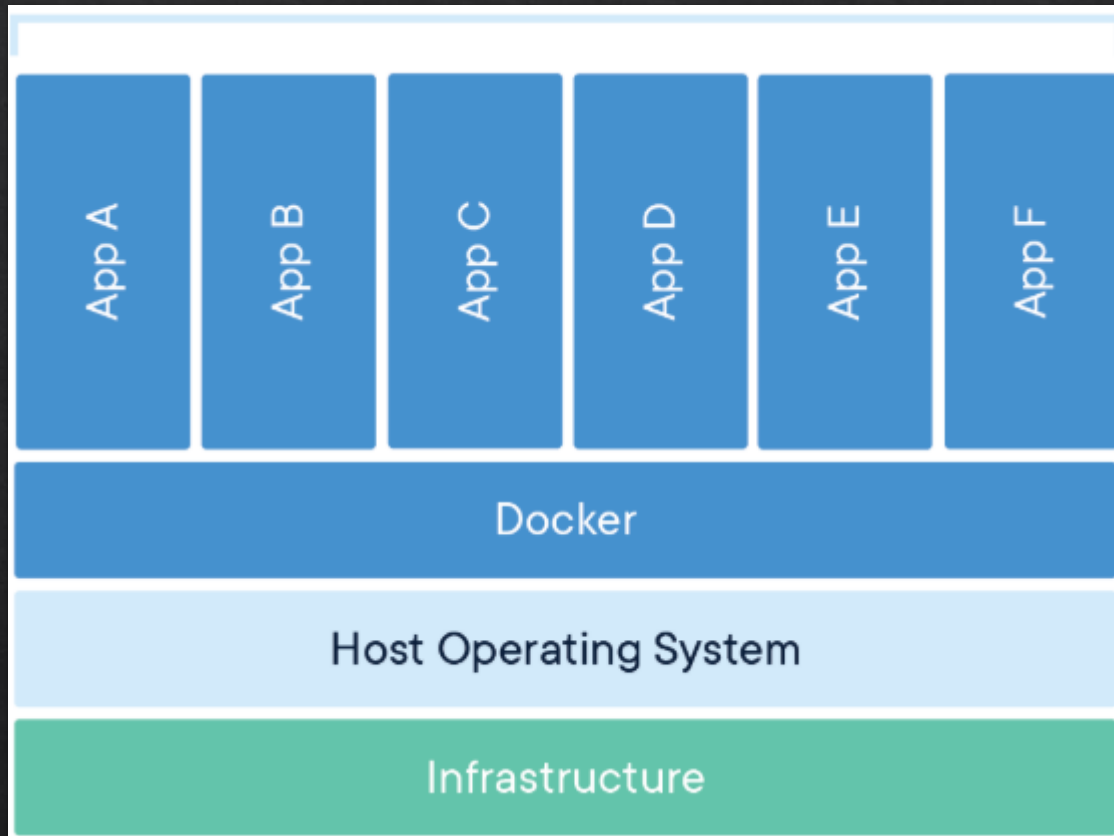
Senior Software Engineer – Virtusa Pvt Ltd

BSc In Computer science

# What is Containerization?

- ◆ Containerization is a software deployment process that bundles an application's code with all the files and libraries it needs to run.
- Packaging application with dependencies
- Runs consistently across environments
- Lightweight and portable
- Faster startup time

This self-contained package, or "container," is lightweight and portable because it doesn't need its own guest operating system. Instead, it shares the kernel of the host operating system while running in its own isolated user space. This isolation means you can run multiple containers on a single host, each with its own set of dependencies, without worrying about conflicts between them.



# Why Do We Need Containerization?

## **Portability**

- ◇ Software developers use containerization to deploy applications in multiple environments without rewriting the program code. They build an application once and deploy it on multiple operating systems. For example, they run the same containers on Linux and Windows operating systems. Developers also upgrade legacy application code to modern versions using containers for deployment.

## **Scalability**

- ◇ Containers are lightweight software components that run efficiently. For example, a virtual machine can launch a containerized application faster because it doesn't need to boot an operating system. Therefore, software developers can easily add multiple containers for different applications on a single machine. The container cluster uses computing resources from the same shared operating system, but one container doesn't interfere with the operation of other containers.

## **Fault tolerance**

- ◇ Software development teams use containers to build fault-tolerant applications. They use multiple containers to run microservices on the cloud. Because containerized microservices operate in isolated user spaces, a single faulty container doesn't affect the other containers. This increases the resilience and availability of the application.

## **Agility**

- ◇ Containerized applications run in isolated computing environments. Software developers can troubleshoot and change the application code without interfering with the operating system, hardware, or other application services. They can shorten software release cycles and work on updates quickly with the container model.



# How does containerization work?

- ◆ Containerization involves building self-sufficient software packages that perform consistently, regardless of the machines they run on. Software developers create and deploy container images—that is, files that contain the necessary information to run a containerized application. Developers use containerization tools to build container images based on the Open Container Initiative (OCI) image specification. OCI is an open-source group that provides a standardized format for creating container images. Container images are read-only and cannot be altered by the computer system.
- ◆ Container images are the top layer in a containerized system that consists of the following layers.

## Infrastructure

- ◆ Infrastructure is the hardware layer of the container model. It refers to the physical computer or bare-metal server that runs the containerized application.

## Operating system

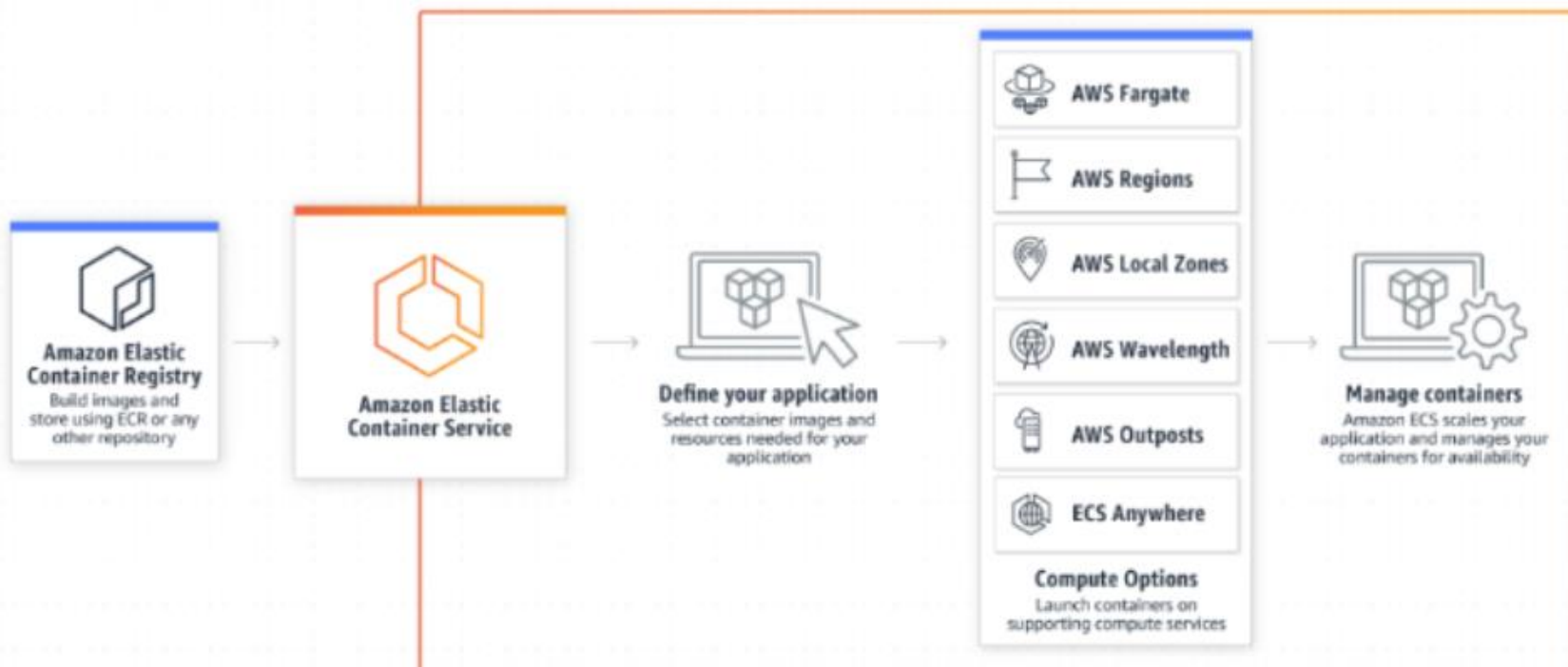
- ◆ The second layer of the containerization architecture is the operating system. Linux is a popular operating system for containerization with on-premise computers. In cloud computing, developers use cloud services such as AWS EC2 to run containerized applications.

## Container engine

- ◆ The container engine, or container runtime, is a software program that creates containers based on the container images. It acts as an intermediary agent between the containers and the operating system, providing and managing resources that the application needs. For example, container engines can manage multiple containers on the same operating system by keeping them independent of the underlying infrastructure and each other.

## Application and dependencies

- ◆ The topmost layer of the containerization architecture is the application code and the other files it needs to run, such as library dependencies and related configuration files. This layer might also contain a light guest operating system that gets installed over the host operating system.



# Virtual Machines vs Containers

What are containers?

- ❖ Containers are lightweight, portable, and self-contained executable images that contain software applications and their dependencies. They are used to deploy and run applications in a consistent way across different environments, such as development, staging, and production. Containers are typically deployed from an image by using an orchestration platform, like Kubernetes. These platforms provide a way to manage and deploy containers at scale.
- ❖ Containers have a number of benefits over traditional virtualization methods. As they are more lightweight and portable than VMs, containers support decomposition of a monolith into microservices. Containers are faster to manage and deploy than VMs, which can save time and money with application deployment.



# What are virtual machines?

- ❖ Virtual machines (VMs) or guests represent instances of an operating system co-located on a physical machine through the use of a hypervisor. Each VM has its own operating system, memory, and other resources, which are isolated from the other VMs on the same physical computer. This allows multiple operating systems to run on the same physical components without interfering with each other.
- ❖ Virtual machines are created and managed using hypervisor software. A hypervisor is software that manages a physical computer's resources and allocates them to virtual machines



- ❖ Containers and virtual machines are similar in function, but containers tend to use less resources and are more flexible.
- ❖ Virtual machines allow multiple applications and operating systems to share the resources of a single physical computer or server and run simultaneously (like Windows and Linux sharing the same computer). A virtual machine carries its files and dependencies, as well as its own copy of the operating system.
- ❖ Containers do not carry a copy of the operating system like virtual machines do. Because of this, the primary difference between containers and virtual machines is that containerization uses compute resources more efficiently. The container runtime is instead installed on the computer or server's existing operating system, allowing all containers to share the same operating system.

## Difference between containers and VMs

- ❖ Virtual machines access the hardware of a physical machine through a hypervisor. The hypervisor creates an abstraction layer allowing the VM to access CPU, memory, and storage. Containers, on the other hand, represent a package that includes an executable with the dependencies it needs to run.
- ❖ This means that each container shares the physical machine's hardware and operating system kernel with other containers.
- ❖ As a result, virtual machines are typically more resource-intensive than containers. However, virtual machines also provide a high level of isolation, which can be important for security and compliance reasons. Containers are more lightweight and portable than virtual machines. This makes them a good choice for applications that need to be deployed quickly and easily, where compute must be optimized.

| Feature              | Container                                                                   | Virtual machine                                                                       |
|----------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Virtualization level | Operating system Level                                                      | Hardware level                                                                        |
| Operating system     | Shares host OS kernel                                                       | Guest OS (full OS instance)                                                           |
| Size                 | Megabytes (MBs)                                                             | Gigabytes (GBs)                                                                       |
| Boot time            | Seconds                                                                     | Minutes                                                                               |
| Resource usage       | Lower                                                                       | Higher                                                                                |
| Isolation            | Process-level                                                               | Hardware-level                                                                        |
| Portability          | Highly portable                                                             | Less portable                                                                         |
| Management           | Typically managed by container orchestration tools (for example Kubernetes) | Managed by hypervisors (e.g., VMware, KVM) or cloud management platforms              |
| Use cases            | Microservices, web applications, CI/CD pipelines, cloud-native applications | Legacy applications, applications requiring strong isolation, diverse OS requirements |

## Architecture of containers versus virtual machines

### **Containers:**

- Share the host OS kernel: Containers leverage the underlying operating system kernel of the host machine.
- Lightweight: Smaller in size and require fewer resources compared to VMs.
- Faster startup: Start and stop quickly due to shared OS kernel.

### **Virtual machines:**

- Run a full guest OS: Each VM has its own dedicated operating system, isolated from other VMs.
- Resource-intensive: Require more resources (CPU, memory, storage) compared to containers.
- Slower startup: Take longer to boot up due to the need to load the guest OS.



## Similarities between containers and virtual machines

- ◆ While containers and virtual machines serve different purposes, they are not mutually exclusive. In fact, they can often be used together to create a powerful and flexible infrastructure.
- Containers are segregated applications or services on a host and share the same operating system
- Virtual machines are isolated instances of an operating system on a physical host and run applications or services
- ◆ Containers have a smaller footprint than virtual machines. This is because containers share the host operating system's kernel, while virtual machines each have their own kernel. As a result, containers can start and stop faster, and they use fewer resources. Virtual machines are isolated from each other, which can help to prevent attacks from spreading between guests.

## Use cases for containers

- ◇ Containers are a popular choice for a variety of use cases, including:
  - **Web development:** Containers are a good choice for web development because they can be easily deployed to a variety of environments, such as development, staging, and production
  - **Microservices architecture:** Containers are a good choice for microservices architecture because they can be used to deploy and manage individual services
  - **Cloud computing:** Containers are a good choice for cloud computing because they can be easily scaled up or down to meet demand
  - **Continuous integration and delivery (CI/CD):** Containers can be used to support automated processes of building, testing, and deploying applications
- ◇ Containers are a powerful tool that can be used to improve application development, deployment, and management. They are a good choice for a variety of use cases, and are becoming increasingly popular in the cloud computing industry.

## Use cases for virtual machines

- ◆ Virtual machines are a popular choice for a variety of use cases, including:
  - **Testing:** Virtual machines can be used to test new software in a safely sandboxed environment. This is because the VM can be easily reset or deleted if the software causes problems.
  - **Development:** Virtual machines can be used to develop software on different operating systems. This can be useful for developers who need to test their software on different operating systems, or for users who want to use different operating systems for different tasks.
  - **Isolation:** Virtual machines can be used to isolate applications from each other. This can be useful for security segmentation and resource partitioning.
  - **Cloud computing:** Virtual machines are a popular choice for cloud computing because they can be easily scaled up or down to meet demand.
  - **Disaster recovery:** Virtual machines can be used in support of disaster recovery strategies. This is because the VM can be easily restored from a backup if the host machine fails.
- ◆ Virtual machines are powerful tools that can be used for various purposes. They are a cost-effective way to run multiple operating systems and applications on the same computer, and they can be used to test new software in a safe environment.

# When to use containers versus virtual machines

| Use Case                                           | Recommendation   | Enterprise business example                                                                                                                            |
|----------------------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Microservices applications                         | Containers       | An ecommerce company uses containers to deploy its microservices-based platform, enabling independent scaling and faster development cycles.           |
| Legacy application modernization                   | Containers       | A financial institution containerizes its legacy applications to migrate them to the cloud, improving portability and reducing infrastructure costs.   |
| Continuous integration/continuous delivery (CI/CD) | Containers       | A software company uses containers to create consistent and reproducible CI/CD pipelines, ensuring reliable deployments across different environments. |
| Security-sensitive applications                    | Virtual machines | A healthcare provider uses virtual machines to isolate sensitive patient data, meeting strict compliance requirements.                                 |
| Running different operating systems                | Virtual machines | A software developer uses virtual machines to test applications on different operating systems (Windows, Linux, macOS) simultaneously.                 |
| Resource-intensive workloads                       | Virtual machines | A media company uses virtual machines to run video encoding and transcoding workloads, taking advantage of dedicated CPU and memory resources.         |



# Types of app containers

At the most basic level, containers come in two types: **system containers** and **application containers**.

## Application containers

- ◆ Application containers are stateless solutions that typically run a single process. When we talk about containerized applications, they are running application containers where each container handles a process, with multiple containers orchestrated by a tool like Kubernetes.
- ◆ Application containers abound in the market. Docker is considered the most popular, though there are several others. Open source development of containers is managed by the OCI.

## System containers

- ◆ System containers—also known as operating system containers—are older and function much like virtual machines, though without the same computing overhead. It behaves as a standalone system and does not include the container image or require specialized software. System containers were typically used in monolithic, legacy applications.

## **Below are some of the common engines and tools types of containerized applications:**

### **Docker**

- ◆ The most popular open source platform for containerization. Docker enables the creation and operation of Linux-based containers.

### **LXC**

- ◆ The open source project of LinuxContainers.org, LXC allows an app to run multiple Linux systems simultaneously, using a single Linux kernel as the operating system.

### **rkt**

- ◆ Also known as Rocket, rkt is an application-based container engine that allows for fine-grained control of containers or as specific components within a Docker container system.

### **CRI-O**

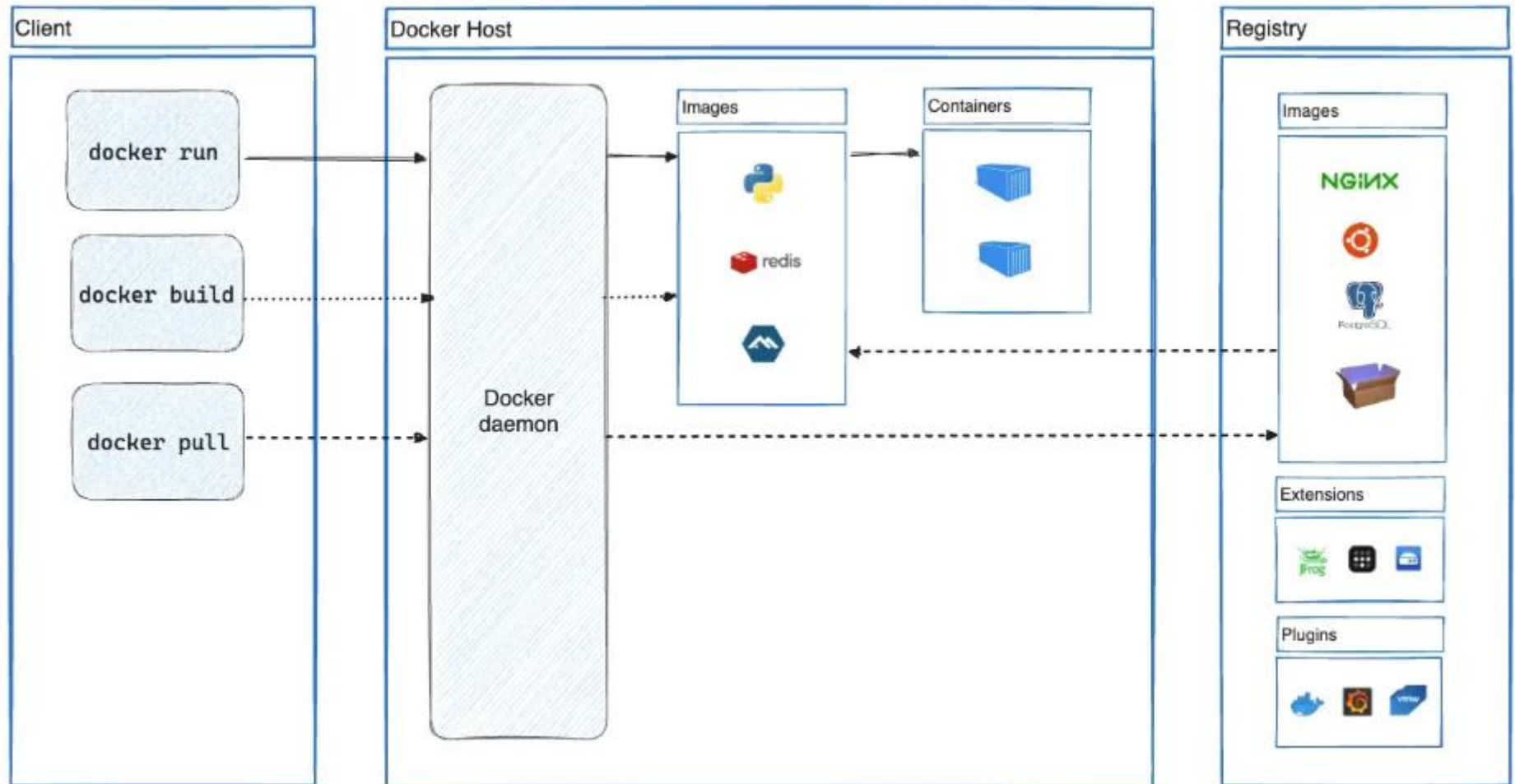
- ◆ A Container Runtime Interface (CRI) for container management platform Kubernetes, used to enable OCI compatible runtimes. Often used as a replacement for Docker when using Kubernetes.

# What is Docker?

- ◆ Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications.
- ◆ By taking advantage of Docker's methodologies for shipping, testing, and deploying code, you can significantly reduce the delay between writing code and running it in production.

<https://docs.docker.com/get-started/docker-overview/#:~:text=Docker%20is%20an%20open%20platform,ways%20you%20manage%20your%20applications.>

# Docker Architecture





- ◇ Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers. The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface. Another Docker client is Docker Compose, that lets you work with applications consisting of a set of containers.

# Docker Image vs Container

Docker Image:

- ◆ • Blueprint
- ◆ • Read-only

Docker Container:

- ◆ • Running instance
- ◆ • Can be started/stopped

# Hands-on: First Container

- ◇ Command:
- ◇ `docker run hello-world`
- ◇ • Pulls image
- ◇ • Creates container
- ◇ • Executes container

# Common Docker Commands

- ◆ • `docker images`
- ◆ • `docker ps`
- ◆ • `docker stop <id>`
- ◆ • `docker rm <id>`



# Benefits of Containerization

- ◆ • Faster deployments
- ◆ • Consistent environments
- ◆ • Easy scaling
- ◆ • Better resource usage

# Thank You

◆ Questions?